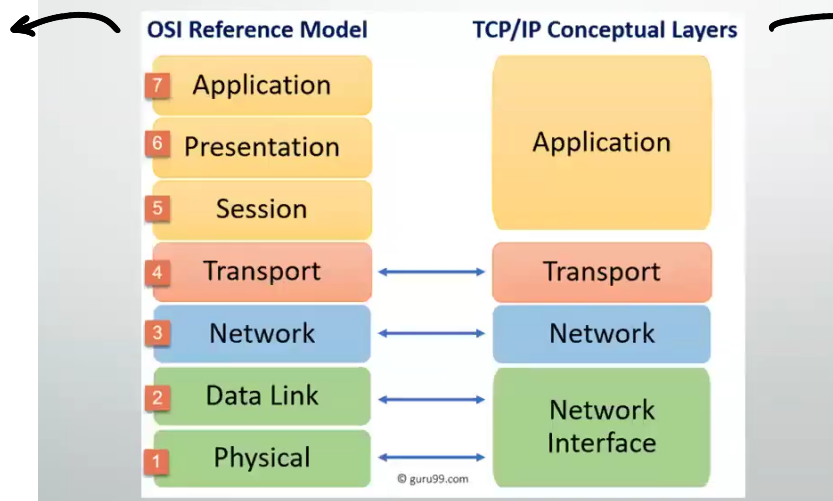


Benefits of splitting communication into layers:

- layers become independent, changes in one layer does not affect the other
- each layer now has defined and specific actions

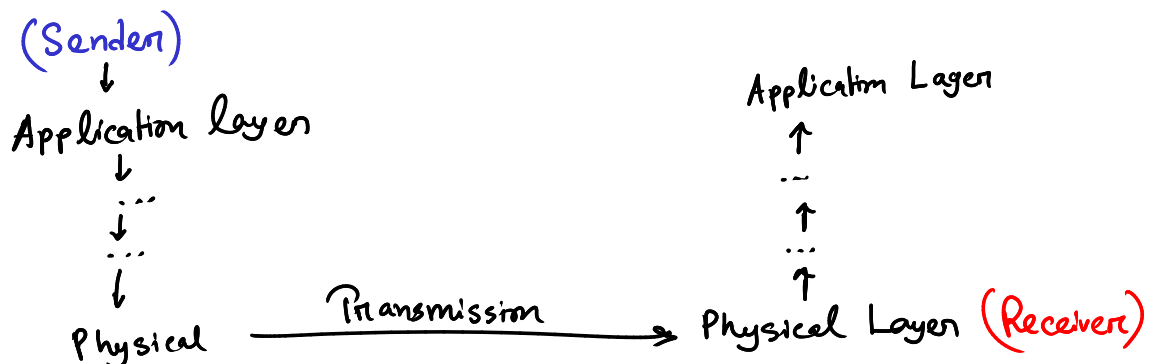
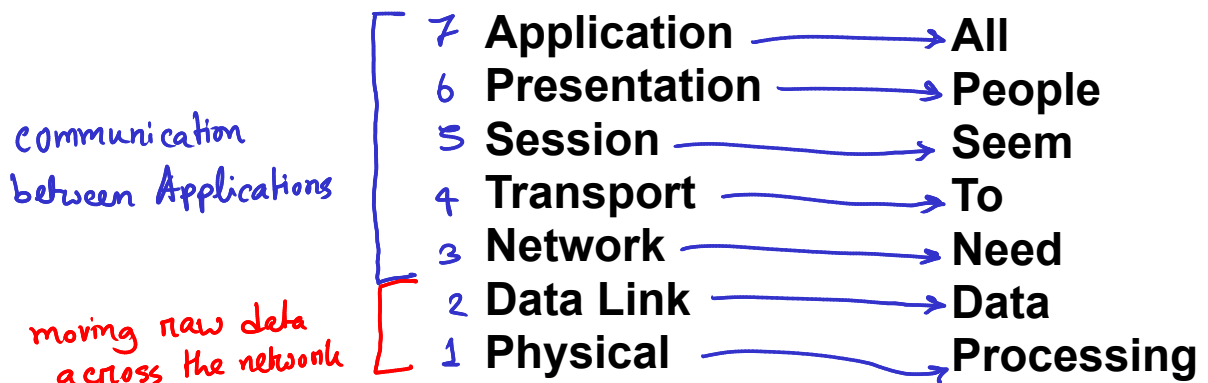
Protocol Suites

De jure
(By law)



De Facto
(By practice)

OSI is a theoretical model that is not used in practice, in practice TCP/IP is commonly used.



(The sender starts at the Application layer and
The Receiver starts at the Physical Layer)

Application layer – provides interface for the user

Presentation Layer – responsible for translation, compression & encryption

Session Layer – handles session, dialog control & synchronization
↳ by restarting sessions that are disrupted or idle
by adding checkpoints in the data

Transport layer – delivers segments across processes.
A commonly used protocol in this layer is TCP

Transport layer breaks down the data received from Session layer into segments which have their own Port Number and sequence number that is used to identify which application it belongs to and what is the order of each segment, after that segments from different applications can be sent altogether, which is how Multiplexing is achieved

Network Layer - delivers packets from sender host to receiver host
commonly used protocol in this layer is IP

Network layer adds logical address and does Routing
IP Address

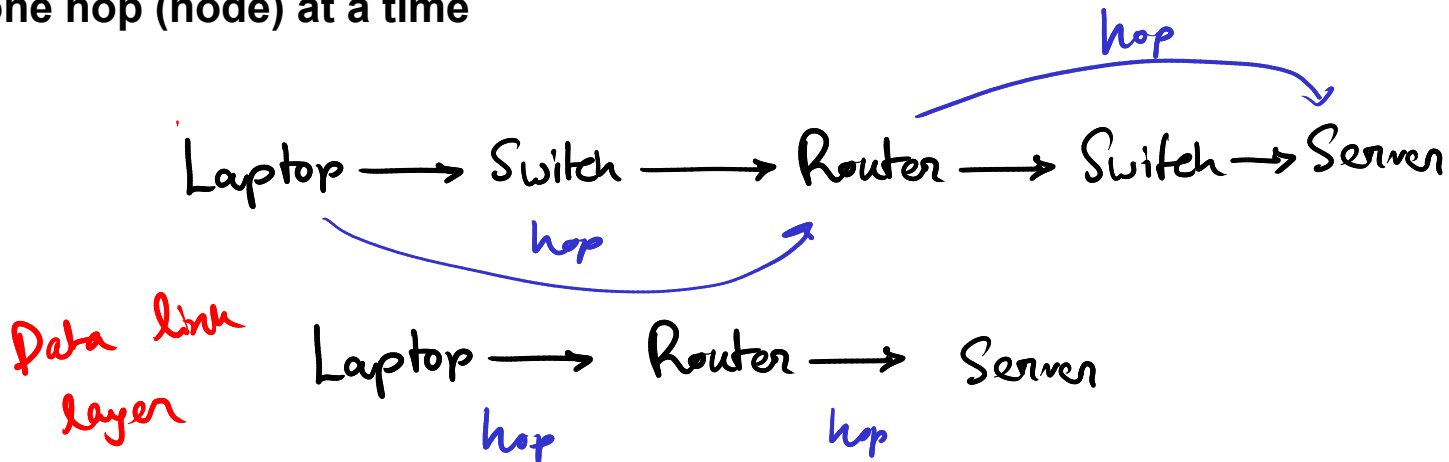
Data Link Layer - delivers frames from one hop to another

(framing is adding physical address as a header and a trailer to the data that it receives from network layer)

MAC Address



* The difference between what Data link Layer and what Network Layer does is that Network layer is responsible for planning and routing from the sender all the way to the receiver, while the Data Link Layer manages one hop (node) at a time



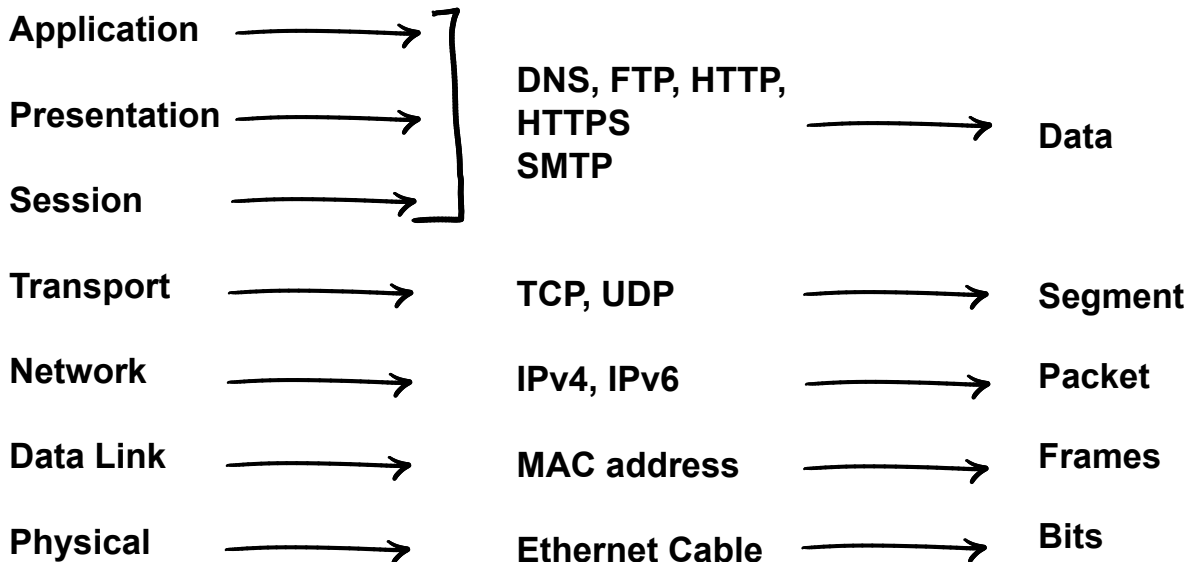
Physical Layer - responsible for transmitting the bits of data by

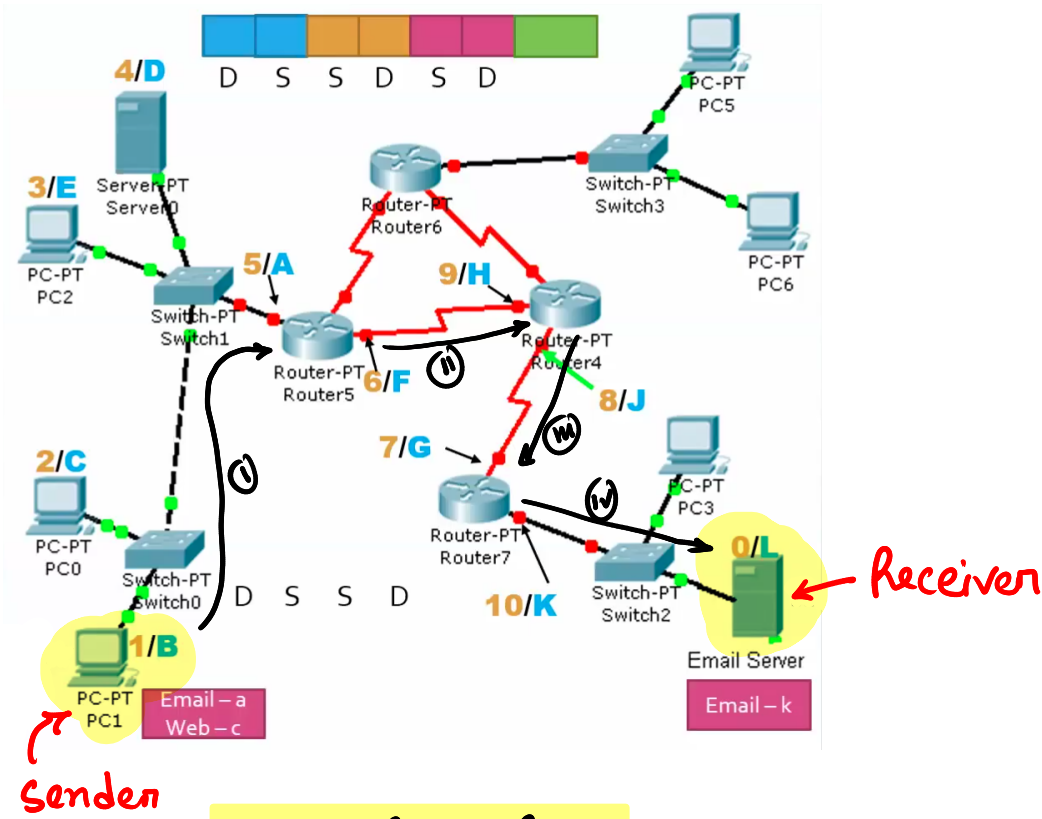
- controlling representation of bits
- controlling data rate
- synchronization

Layers

Protocols

PDU





Transport layer

Source	Destination
a	k

Network layer

Source	Destination
1	0

Data link layer

	senden	receiven
(i)	B	A
(ii)	F	H
(iii)	J	G
(iv)	k	L

Application Layer

Two main Methods:

- i) Client-Server
- ii) P2P (Peer to Peer)

Webpage:

- contains a base HTML file (main object)
- html file contains reference to objects (other html files, jpeg images etc)

HTTP Request message:

Format:

Request Line : method URL Version

Header lines : header field name: value \r\n

Entity Body : Optional format for sending data (POST, PATCH)

example:

```
Request line → GET //somedir/index.html HTTP/1.1\r\n
Header lines → Host: www.someschool.edu\r\n
               User-Agent: Firefox/3.6.10\r\n
               Accept: text/html,image/gif, image jpeg\r\n
               Accept-Language: fr \r\n
               Connection: close\r\n
end of headers → \r\n
Entity Body → Entity Body
```

non-persistent

keep-alive → persistent

Methods: GET, POST, HEAD, PUT, PATCH, DELETE

HTTP 1.0

HTTP 1.1

HTTP Uploading Format:

i) **POST Method**: uses the Entity body

ii) **URL Method**: Uses **GET Method**, sends data in the URL (**google searches**)

HTTP Response message:

Format:

Request Line : **Version** **Status Code** **phrase**

Header lines : **header field name** : **value** **\n\n**

Requested HTML File : **data**

HTTP Connection Types:

i) **Persistent**

ii) **Non-Persistent**

- **Initiate TCP connection** (RTT) ⁽ⁱ⁾
- **Request base file** (RTT+FTT) ^{(ii) (iii)}
- **Terminate TCP connection** (RTT not counted)

- **Initiate TCP connection** (RTT)
- **Request another object** (RTT+FTT)
- **Terminate TCP connection** (RTT not counted)

∴ for x objects

Total
Response time = $RTT \times \underset{(i)}{1} + HTTP\ RTT \times \underset{(ii)}{x} + FTT \times \underset{(iii)}{x}$

FTT - File Transmission Time

- **Initiate TCP connection** (RTT) ⁽ⁱ⁾
- **Request base file** (RTT+FTT) ^{(ii) (iii)}
- **Request another object** (RTT+FTT)
- **Request another object** (RTT+FTT)
- **Terminate TCP connection** (RTT not counted)

Total
Response time = $RTT \times \underset{(i)}{1} + HTTP\ RTT \times \underset{(ii)}{x} + FTT \times \underset{(iii)}{x}$

☐ Total Objects = 15, size = 10MB, client download speed = 8Mb/s,
persistent HTTP, TCP client->server time = 15ms,
HTTP server->client = 20ms. Find the Total Response Time

$$\text{Total RTT} = \text{RTT} \times 1 + \text{HTTP RTT} \times 15 = (15 \times 2) \times 1 + (20 \times 2) \times 15 = 630\text{ms}$$

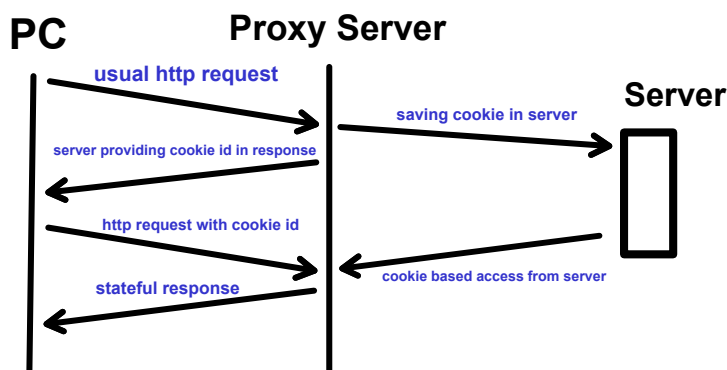
$$\text{Total FTT} = (15 \times 10 \times 8) \text{bits} / 8 = 150\text{ms}$$

non-persistent

$$\therefore \text{Total Response Time} = \text{Total RTT} + \text{Total FTT}$$

Cookies:

- makes HTTP Requests **stateful**. Stateful requests utilizes user's previous actions to give better feedback
- after the first HTTP request a cookie is generated in a **cookie server**
- the cookie is sent through the header of the HTTP Response from the server
- the cookie is then stored in the browser, but also exists in the cookie server.
- from then on it is carried with the header in HTTP Requests to the server
- cookies are different for each browser, and they do not expire unless browser memory is cleared, then new cookies are needed
- a single server has unique cookie ids, but the cookie ids of different servers can be the same



Email & DNS

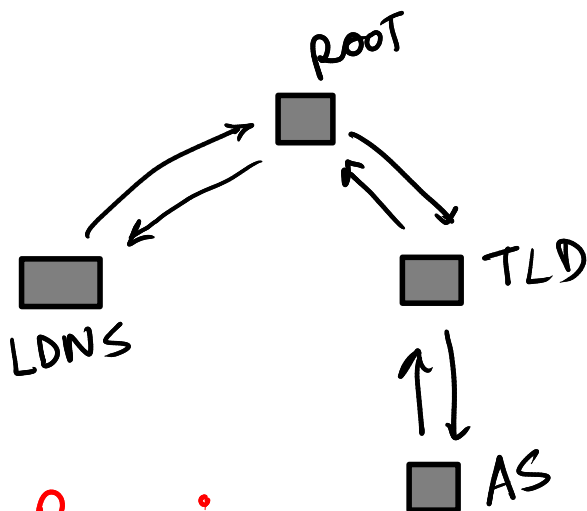
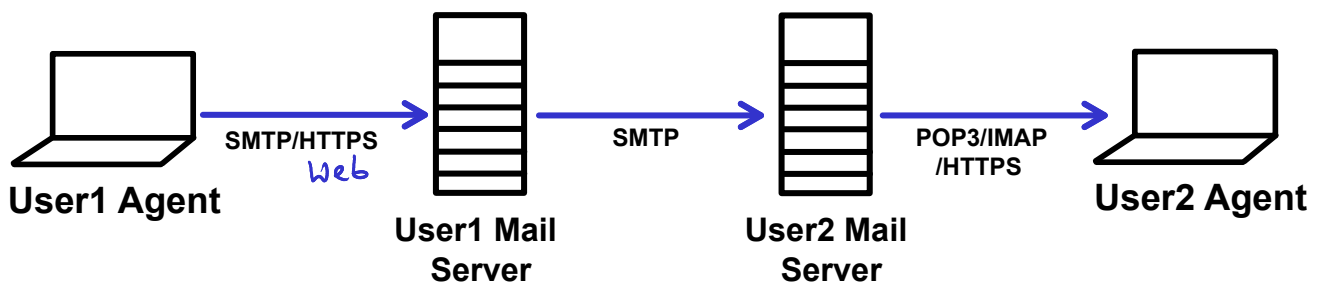
main components of Email:

- user agent: software
- mail server: mailbox, message queue
- protocol: SMTP, HTTPS

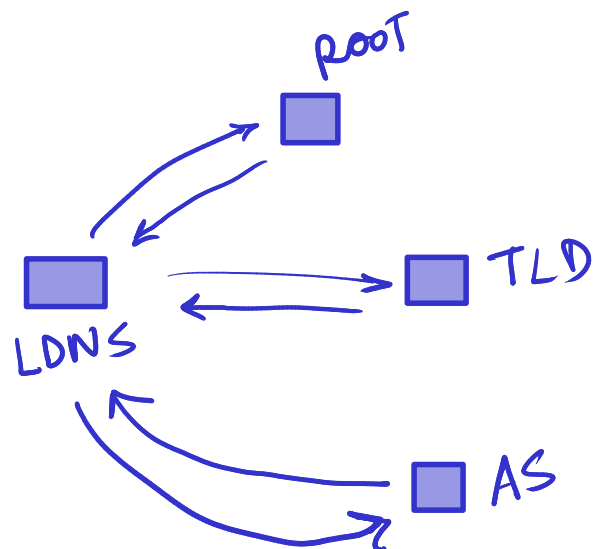
user agent can be software (SMTP)
on Webmail (HTTPS)

mailbox: stores mails (unique to every user)

message queue: queue of outgoing mails



Recursive
Resolution



Iterative
Resolution

TCP Congestion control

can be done in two ways:

- ① Open loop
- ② Closed loop (Feedback)

When congestion happens?

load > capacity
↳ number of packets the network can handle

number of packets sent to the network

processing rate

departure rate

Arrival rate

processing rate

routers have input and output queues

↳ in these cases the queues keep getting longer and longer

when a packet is delayed and no ACK is received the packet is resent making the delay of congestion worse

Target: control congestion while keeping load below capacity, not too far below, then the maximum bandwidth is not utilized

types: Open loop (prevention)
Closed loop (Removal)

Back Pressure
Choke Packet
Implicit Signaling
Explicit Signaling

Retransmission Policy
Window Policy
Acknowledgement Policy
Discarding Policy
Admission Policy

Open Loop:

Retransmission Policy: Delaying Retransmission helps reduce congestion

Window Policy: Selectively Repeating window instead of Go Back N reduces congestion

Acknowledgement Policy: Not Acknowledging every packet reduces congestion, that is why ACK is sent once every few packet received

Discarding Policy: Router can sometimes drop less important packets to prevent congestion

Admission Policy: Preventing no new virtual circuits can prevent congestion

Closed Loop:

Back Pressure: Congested node(Router) stops receiving data from upstream (previous) node. The upstream node eventually gets congested and does the same, in this way the sender is informed of congestion along the road and reduces data transmission. (Only possible in Virtual Circuits)

Choke Packet is sent from the router directly to the sender and the sender reduces data transmission. It is sent back using upstream nodes but those nodes take no action for congestion

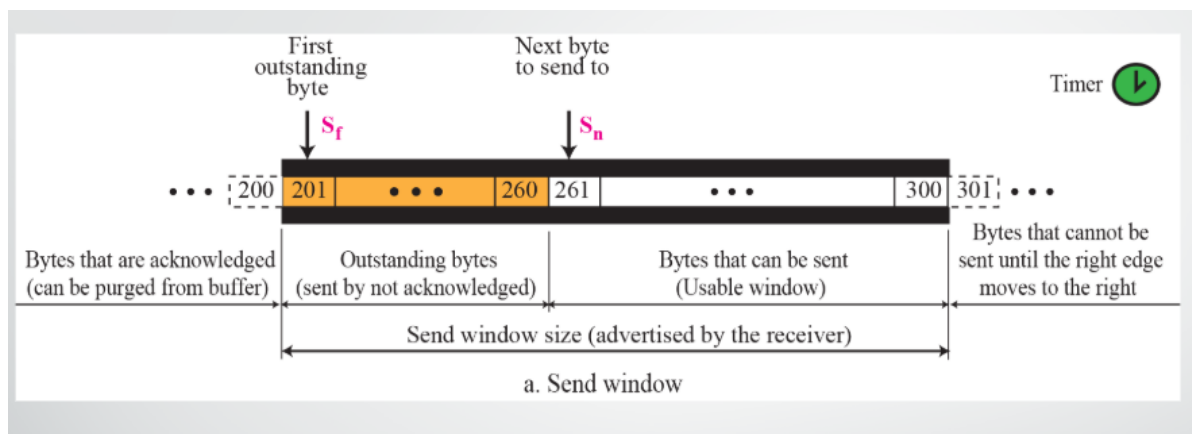
Implicit Signaling: The source may realize there's a congestion by the delayed acknowledgement and sometimes none, then the source slows down

Explicit Signaling: No separate Choke Packet or anything is sent, but signals included in the data packet is sent forward or backward to warn the source or receiver

Waiting for ACK

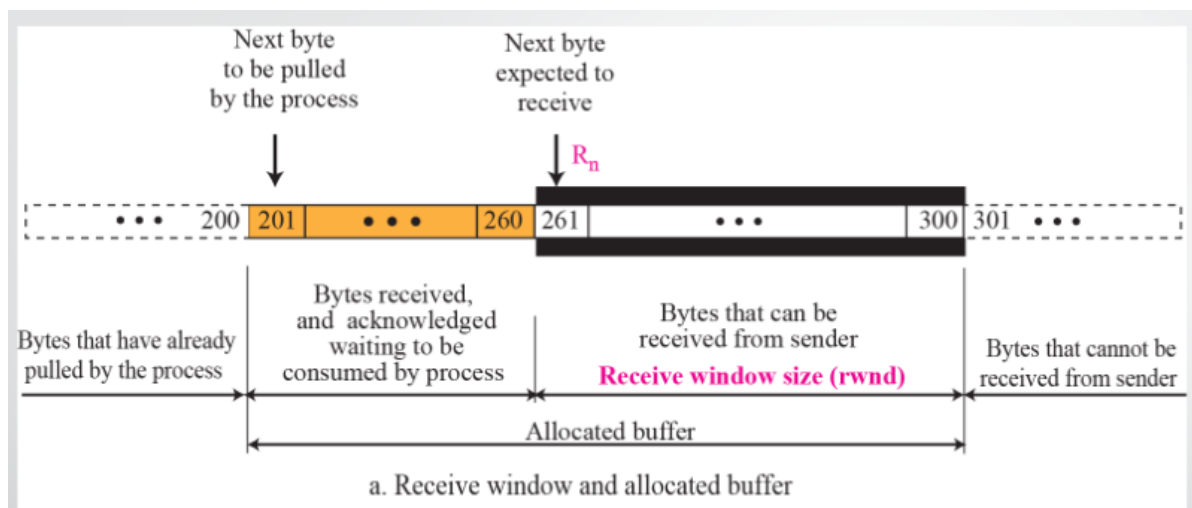
When one packet has missed transmission





Sf is the first byte that was sent but not acknowledged, this is updated with the latest acknowledgement received, after acknowledgement is received the whole section is discarded from the Sender Window

Sn is the next pending byte to be sent



Rn is the request, if bytes before Rn is received, the receiver will request from Rn

when the acknowledged bytes are processed by the application, it frees up the receiver window.

Congestion Window: **cwnd**

Receiver Window: **rwnd**

TCP Window size = $\min(\text{cwnd}, \text{rwnd})$

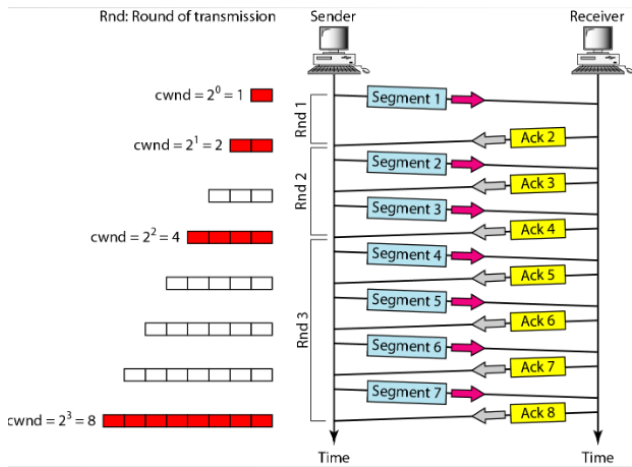
just like bottlenecking

TCP Congestion Control:

1. Slow Start: Congestion window starts with one **Maximum Segment Size(MSS)** and then increases exponentially with each acknowledgements
assumption: $rwnd > cwnd$

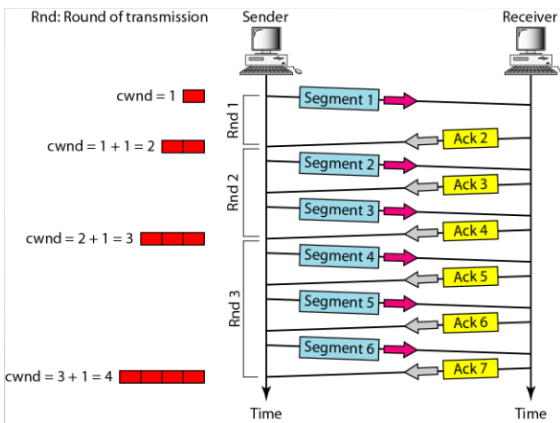
each segment 1 byte
each segment acknowledged individually

The congestion window increases exponentially until reaching a threshold **ssthresh**



increased to 8 due to
8 ACK
exponential every RND (RTT)

2. Congestion Avoidance: Congestion window increases by 1 for ACK received for all packets in each RTT, not by 1 for every individual ACK like in Slow Start. This increase is linear until Congestion is detected



increased cwnd every RND after
all ACK of that round is received

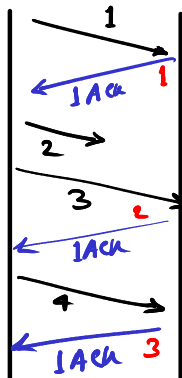
linear across RNDs

3. Congestion Detection: Depending on the type of detection we fall back to one of two detected phases:

- i) detection by time-out (**Severe congestion**): fall back to (1) Slow start
- ii) detection by three duplicate ACK (**less Severe**): fall back to (2) Congestion Avoidance

Ack timeout
Ack not sent by receiver

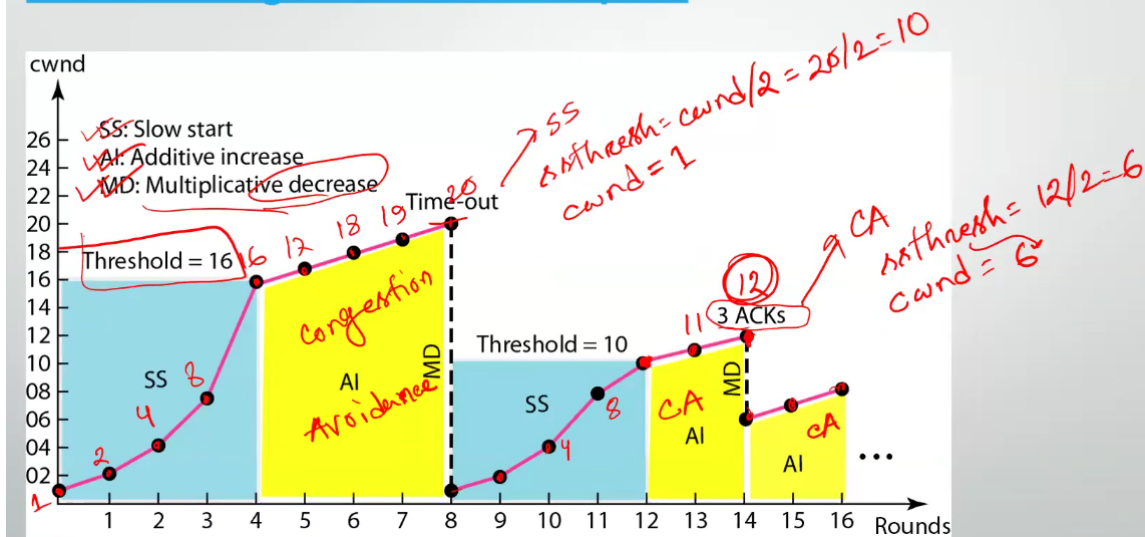
One missing packet
3 ACK from previous packet



Congestion Detection:

- Congestion detected by **time-out (severe congestion)**
 - starts new slow start phase
 - $ssthresh = \max(2, \text{floor}(cwnd/2))$
 - $cwnd = 1$
- Congestion detected by three **ACKs (less severe congestion)**
 - starts new congestion avoidance phase
 - $ssthresh = \max(2, \text{floor}(cwnd/2))$
 - $cwnd = ssthresh$

TCP Congestion Example:



ACK is what is requested from receiver
SQN is what is being sent